Every element knows who made it, why, and whether you're allowed to ship it.

Strata is a shared design canvas with one AI teammate that lives in the file from the first frame. It remembers who decided what and why, learns the team's taste from what people keep and undo, and lets the team try and buy design assets directly on their own design, in the app or from an ASI:One chat.

The problem

Today's AI assistants are built for one person, but design is done by teams. When a team shares an AI, it forgets, mixes people up, or silently picks a side. Each row below is a limitation that the labs themselves or recent benchmarks have published.

LIMITATION	EVIDENCE	WHAT STRATA DOES
Team AI has no safe memory	ChatGPT group chats turn memory off entirely; Claude isolates memory per project, not per person	Every memory is attributed to a person and tied to the canvas elements it is about
Shared memory loses who said what	GroupMemBench (2026): best system 46% overall, 27% on updated facts	Author and on-whose-behalf are stored with every change, never summarised away
Conflicting teammates break the agent	Multi-User LLM Agents (2026): models don't keep stable priorities across users	A conflict card names both people instead of silently choosing
Long context rots and costs grow	Anthropic, context engineering (2025): recall degrades as the window fills; compaction is lossy	Each request gets a small, budgeted context instead of the full history
People can't see what the AI did	Design tools show agent chats per file, not per layer; no element-level human/AI history	Click any element to see its full lineage and the reason behind each change

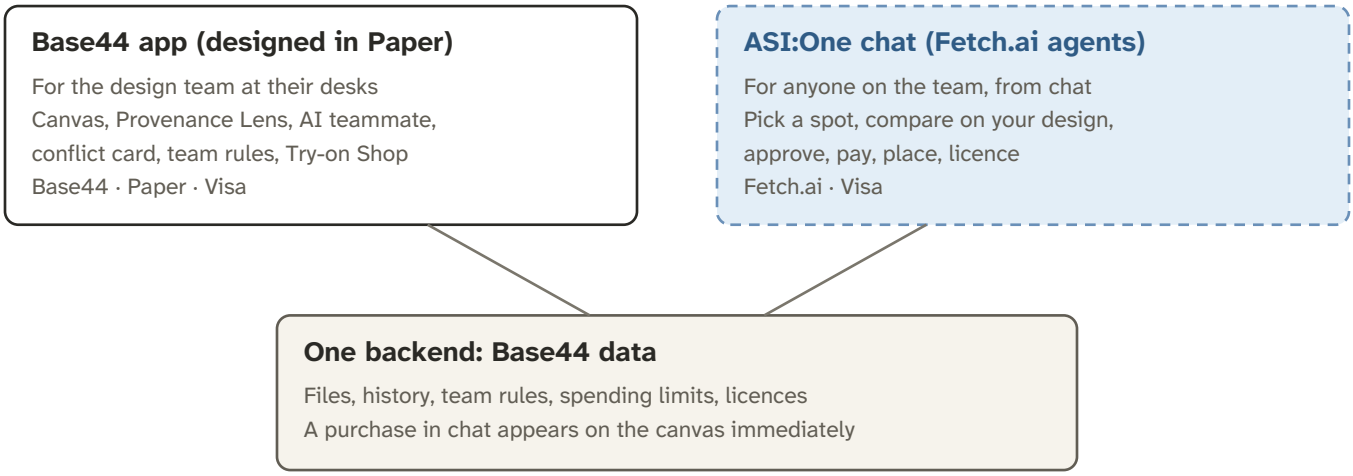
The shopping pain

Outside Adobe's and Canva's own stock libraries, designers buy fonts, icons and illustrations in store tabs that know nothing about their project or team. They judge an asset on a generic preview, pay, and only then see whether it fits. Licence knowledge lives in someone's inbox, so a personal-use asset can quietly end up in client work. Strata turns the canvas into the store.

Who it is for: small product design teams (2 to 8 people), and the independent creators who sell fonts, icons and illustrations to them.

What we are building

One product, two surfaces, one backend. The team designs in the Base44 app; anyone can get things done about the design from an ASI:One chat. Both read and write the same data.



Features

FEATURE	WHAT THE USER SEES	WHY IT EXISTS
Shared canvas	Frames, text, images and shapes; teammates' changes appear when a drag or edit ends	The surface everything else hangs on
Provenance Lens	Click an element: who changed it, human or AI, on whose request, the prompt, the AI's stated reason, before and after	Element-level human/AI history is the core novelty
AI teammate	Chat that edits the canvas and explains each edit; "Catch me up" shows what changed since you last looked	The AI lives in the file from day one
Team taste memory	Learned rules ("illustrations are flat, two colours") with who taught them; team, personal or split; editable	The AI remembers for future outputs; people stay in control
Conflict guard	Before the AI undoes a teammate's deliberate choice, a card names both people	No silent side-picking
Try-on Shop	Assets ranked by team taste, previewed watermarked on the selected element, bought in the file	Shopping on your real design, not a stock thumbnail
Spending limits and licence ledger	Per-person monthly limit; intent, cart, payment, receipt trail; each licence tied to the elements that use it; export guard	Trust: nothing charged without approval or beyond a limit
ASI:One shopping agent	The whole shop-and-place job done in chat with seven Interactive Cards and in-chat payment	Decide and act without opening the editor

The method: LACE

LACE (Lineage-Anchored Context Engine) keeps a team's design memory small, attributed and conflict-aware. The unit is a memory *atom*: one edit, prompt, AI rationale, rule, purchase or conflict, stored with its author, on whose behalf, the elements and properties it touches, and a one-line summary.

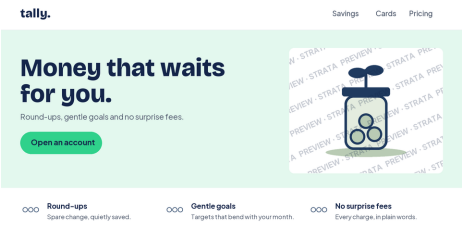
STAGE	WHAT IT DOES	WHY IT MATTERS
1 · Anchor	Logs every human gesture and AI edit as an atom. The AI must state its intent before acting. A cheap model writes each summary once.	Provenance for free; the cost is paid once, not on every request
2 · Weigh	Learns preferences from what people keep and undo, per member, as smoothed counts. Two members agreeing makes a team rule; disagreement becomes a conflict.	Taste is learned from behaviour, not written by hand, and every rule traces to its evidence
3 · Pack	For each request: elements near the selection, locks, open conflicts, matching rules, and what this person hasn't seen. Scored, then packed into a fixed token budget.	Bounded cost however long the history; locks and conflicts are never dropped

Honest framing: LACE is an engineered retrieval and packing method, not a learned model. What is new is anchoring memory to design elements, attributing it to people, and learning team versus personal taste from implicit keep and undo signals.

The ASI:One agents (Fetch.ai)

Five agents registered on Agentverse. The Concierge runs a LangGraph plan with ASI:One as the model; every card is a pause, every click resumes the plan, and failures have recovery paths.

AGENT	ROLE
Concierge	Chat Protocol and Payment Protocol. Plans the steps, sends the seven cards, recovers from failures, never charges twice.
Curator	Ranks the catalog against the team's learned rules; ASI:One writes a one-line reason.
Renderer	Draws the real design with each option in place, watermarked, as the card images.
Checkout	Checks the spending limit, opens Stripe checkout, verifies payment before delivery.
Ledger	Places the asset on the file and records the licence on the elements that use it.



The chat flow: 1 pick the spot (image tiles from the file) · 2 confirm the brief (prefilled from team rules) · 3 compare options placed on your design · 4 look closer, pick a licence · 5 approve against your spending limit · 6 pay in the chat · 7 receipt: placed, licensed, logged, with the next fix suggested.

Card image rendered by the Renderer agent: an option tried on the team's own hero, watermarked.

Tracks and challenges this idea completes

Each challenge is judged separately, so one project can win several. Status is as of Oct 17, evening.

CHALLENGE	PRIZE	HOW STRATA MEETS IT	STATUS
Fetch.ai · ASI:One Agent Challenge (main target)	$1,500 / $1,000 / $500	Five agents on Agentverse; Chat Protocol; whole workflow inside ASI:One; all five card types; Payment Protocol with a 12% marketplace fee; LangGraph plan with retries and recovery; live data from the file	**Built, tested locally**
Visa · The Future of Shopping	$2,000	Discovery by learned taste, decision on your own design, checkout in app and in chat, spending limits, intent-cart-payment-receipt trail, licence tied to elements. Stripe test mode with a Visa test card	**Flow designed; Stripe next**
Base44 · Build Something People Want	$1,500 / $500	The product is built in Base44: canvas, history, team AI, shop, checkout and all data. Real users tested on site; agents use the app as their backend	**Schemas and API ready**
Paper · Best Designed Hack	$1,500 / $1,000 / $500	Designed by hand in Paper: "drafting table" system (AI drafts in dashed blue pencil, people ink), seven screens and a card storyboard; shipped as the Base44 app	**Designs done; build next**
LA Hacks main track · Applied Intelligence	Track prize; overall $2,000 / $1,500 / $1,000	AI for a real workflow: team memory, provenance and trustworthy agent commerce, with a working demo	**Demo and video**

Requirement fit, in one line each

SPONSOR RULE	FIT
Fetch: primary workflow completes inside ASI:One, no custom frontend	Yes. The chat flow stands alone; its receipt card shows the placed design. The Base44 app is a second surface.
Fetch: public GitHub with run instructions; agent profiles and chats via the Submission Agent	Repo and README ready; profiles and shared chats once agents are live
Base44: built substantially in Base44, live working app	Yes. All data, rules, payments and UI in Base44; agents are thin clients of one Base44 function
Visa: innovation, UX, technical, commerce impact, trust (no required API)	Yes. Trust is designed in: approval before charge, spending limits, explainable picks, audit trail
Paper: design fundamentals, ease of use, personal craft	Yes, if the Base44 app is built from the Paper screens rather than default styling

No two challenges conflict.

Paper is the design, Base44 is the app, Fetch.ai is the chat agent, and Visa is the shopping and payment experience across both. The Fetch entry does not depend on Base44; if the app slips, the agents still run on a stand-in backend and the Fetch submission is complete.

Where it stands and what's next

Built and verified

PIECE	EVIDENCE
UI design	Paper file: design system plus seven desktop screens and the ASI:One card storyboard
Five Fetch agents	Full chat flow runs end to end over real agent messaging with the ASI:One model; three automated tests (happy path, over-limit, rejected payment) pass
Card images	Renderer draws the team's real page with each option placed, watermarked
Base44 contract	Entity schemas, the agent API function, seed and check scripts written

Next, in order

STEP	WHAT	UNBLOCKS
1	Connect the agents to Agentverse; test cards and payment in ASI:One; share chats	Fetch.ai
2	Get the Base44 Builder plan; create entities and the agent API; seed data	Base44, Visa
3	Build the app from the Paper screens: canvas, Lens, teammate, conflict card, shop	Base44, Paper, Visa
4	Point the agents at Base44 so a chat purchase appears live on the canvas	All
5	User tests on site, 3-minute video, Devpost, Fetch Submission Agent	All

Open risks

RISK	FALLBACK
Base44 needs the paid Builder plan	Kickoff credit code, or one month at the student discount
ASI:One payment sheet may not render	In-chat "send a link" to Stripe; verified before delivery
Image-tile picker card is untested in ASI:One	Swap to a carousel card in minutes
Time	Cut order: rule recomputation, export guard, in-app checkout, realtime sync. Never the Lens, the conflict card, or the chat purchase